MALWARE FAMILY CLASSIFICATION

Using a Custom CNN with Attention

TASK 2 REPORT

CNN Baselines, the Proposed Custom CNN + CBAM Model, and First Results

Dataset: Malimg Original
Track: Custom CNN + Attention
Course: ICE478
Group: Group02

Submitted by

Name	ID
Sadman Tayef	2022-3-50-002
Shenila Sharior	2022-3-50-009
Mehjabin Tuli	2022-3-50-027
Fihima Tajrian	2023-1-50-006

1. Introduction

Task 1 established that the Malimg Original dataset contains malware binaries rendered as grayscale byte images, organized into 25 families, with severe class imbalance and a non-trivial number of exact duplicate images. Task 1 also identified a research gap: existing CNN + attention studies on Malimg rarely isolate the independent contribution of attention from other simultaneous changes such as an added classifier, a second modality, or a pretrained backbone.

Task 2 builds directly on those findings. It finalizes a leakage-free data pipeline, trains four representative CNN baseline, designs, implements, and evaluates the proposed model a custom CNN with a Convolutional Block Attention Module (CBAM). So that the effect of attention can be measured against an identical no-attention backbone. This report follows the same sequence in which the two Task 2 notebooks (baselines and proposed model) were executed.

2. Data Integrity Pipeline

Task 1 flagged 260 exact-duplicate file groups (895 redundant files) using byte-level SHA-256 hashing of the raw PNG files. The Task 2 notebook repeats this check at a stricter level: instead of hashing the raw file bytes, it hashes the fully decoded pixel content (grayscale pixel array plus dimensions) of every image. Decoding before hashing catches images that are visually identical but stored with different PNG compression settings, which raw byte-hashing can miss. Table 1 summarizes the resulting, stricter integrity pass.

Table 1: Stricter data-integrity audit repeated for Task 2, using decoded pixel-content hashing instead of raw-file hashing.

Check	Result
Raw dataset files	9,339
Raw malware families	25
Unique decoded images	8,019
Redundant copies removed	1,320
Cross-family duplicate groups	0 (no label conflicts)
Families unable to support an independent split	Yuner.A (800 raw files → 1 unique decoded image)
Final modeling images	8,018
Final evaluated families	24

Two decisions follow directly from this audit and resolve the open items from the Task 1 review. First, duplicate copies are removed before the train/validation/test split, keeping exactly one representative image per unique decoded sample this is the leakage-prevention decision that Task 1 left open. Second, the Yuner. A family is excluded from modeling: all 800 of its files decode to a single unique image, so it cannot support an independent train/validation/test split without leaking that same image across partitions. This is why the modeling pipeline works with 24 evaluated families instead of the 25 raw folders reported in Task 1.

2.1 Stratified Split and Leakage Gate

The 8,018 deduplicated images were split 70% / 15% / 15% into training, validation, and test sets using stratified sampling on the class label, so that every one of the 24 families is represented in every partition. Resizing (224 × 224, RGB) is applied inside the tf.data pipeline after this split, so no partition is affected by another. A dedicated integrity gate re-checks, after splitting: (a) zero filepath overlap between partitions, and (b) zero exact pixel-content-hash overlap between partitions. Both checks returned zero overlap, so the split is leakage-free by construction, not only by intention.

3. Preprocessing and Training Setup

The same preprocessing and training configuration is reused for every baseline and for the proposed model, so that comparisons in Section 6 isolate architecture rather than pipeline differences.

Table 2: Shared preprocessing and training configuration used for every baseline and the proposed model.

Setting	Value
Input size	224 × 224 × 3
Batch size	32
Split	70% train / 15% validation / 15% test
Augmentation (training only)	Random translation (±2%), random zoom (±5%), random contrast (±5%)
Imbalance handling	Class weights computed with sklearn 'balanced' scheme, applied only to training loss
Optimizer / loss	Adam / Sparse Categorical Crossentropy
Initial learning rate	1e-3 (baselines), with ReduceLROnPlateau
Max epochs / early stopping	25 epochs, early stopping on validation loss
Primary metric	Macro-F1
Random seed	42, fixed across NumPy, Python, and TensorFlow

Macro-F1 is used as the primary comparison metric, and not plain accuracy, precisely because of the imbalance documented in Task 1 (Allaple.A has 2,949 raw images versus 80 for Skintrim.N): accuracy alone would be dominated by the large classes. Weighted F1 and per-class recall are reported alongside it for a complete picture.

4. Baseline Models

Four representative baselines were trained under the identical pipeline above: a compact custom CNN, and three ImageNet-pretrained backbones (ResNet50, DenseNet121, EfficientNetB0) used with frozen convolutional weights and only the classification head trained. Freezing the pretrained backbones keeps the baseline experiment computationally comparable to training a small custom network from scratch, and is the reason the ResNet50 baseline trains in under three minutes.

4.1 Baseline 1 - Simple CNN

The Simple CNN uses three convolutional blocks, global average pooling, a 128-unit dense layer with 0.40 dropout, and a 24-way softmax output. This exact backbone and classifier head is reused unmodified as the no-attention counterpart of the proposed CBAM model in Section 5, which is what makes the attention-effect comparison in Section 7 fair.

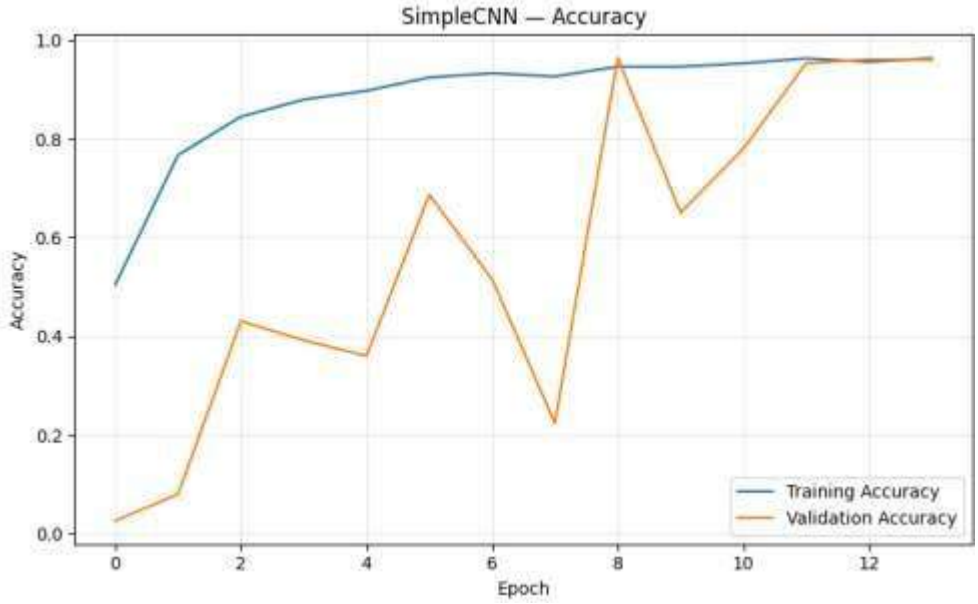

Figure 1: Simple CNN training/validation accuracy curve.

Validation accuracy is volatile for the first eight epochs, dipping as low as 22% around epoch 7, because the balanced class weights push the model to correct rare-class mistakes before it has seen enough of those classes to do so reliably. From epoch 8 onward both curves converge and track each other closely above 95%, showing the model generalizes well once this early adjustment period passes.

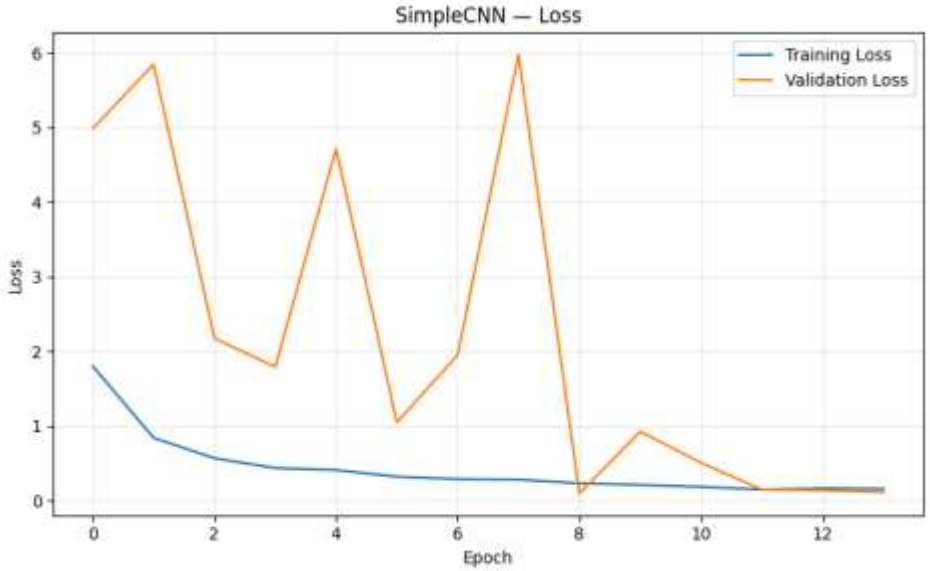

Figure 2: Simple CNN training/validation loss curve.

The loss curve mirrors the accuracy curve: a sharp validation-loss spike around epoch 7, peaking near 6.0, coincides exactly with the accuracy dip in Figure 1, then falls sharply and settles under 0.2 alongside the training loss from epoch 11 onward, the point where early stopping's patience window found a stable, low-loss checkpoint to restore.

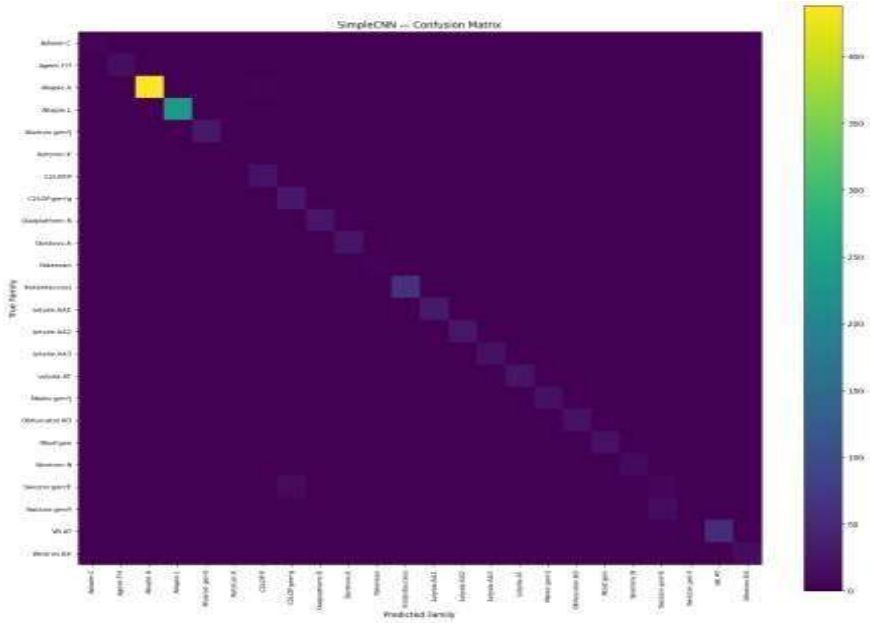

Figure 3: Simple CNN confusion matrix

The matrix is strongly diagonal even for this lightweight model, with almost all predictions falling on the correct family. The largest classes, Allaple.A and Allaple.L, show the brightest diagonal cells simply because they contain the most test images; the handful of visible off-diagonal cells are concentrated around a few families with similar raw byte texture, which the CBAM model in Section 5 reduces further.

4.2 Baseline 2 - ResNet50

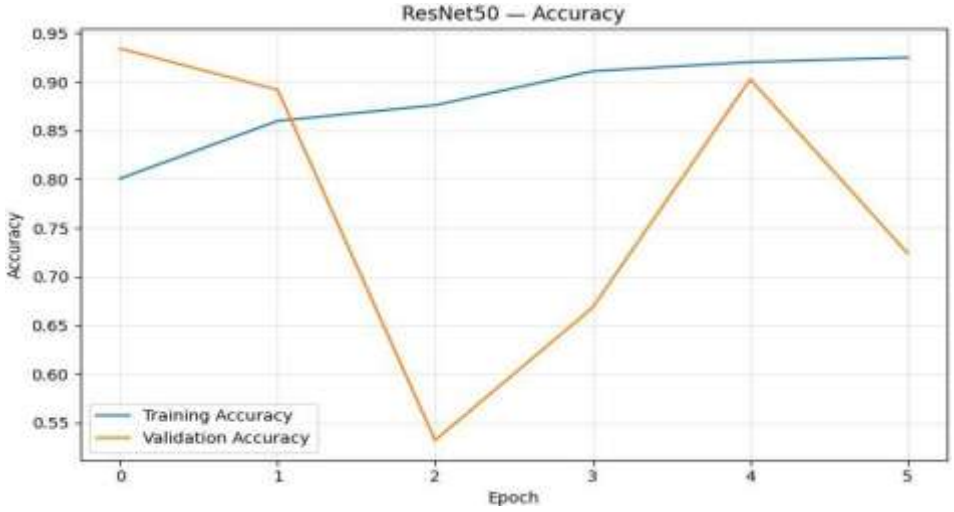

Figure 4: ResNet50 training/validation accuracy curve.

ResNet50 converged fastest in wall-clock time (2.95 minutes, 6 epochs before early stopping) but produced the weakest Macro-F1 of the four baselines (80.79%). The validation curve swings sharply between epochs, dropping to 53% at epoch 2 before recovering, which is a symptom of the frozen ImageNet backbone struggling to separate malware textures consistently from one epoch to the next.

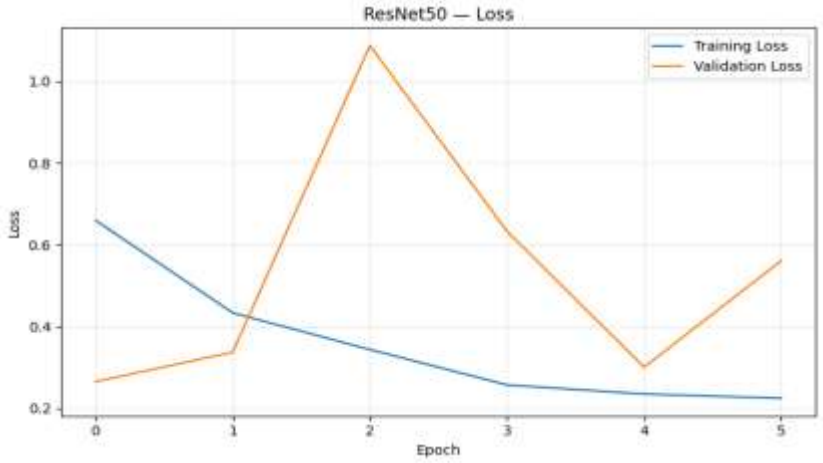

Figure 5: ResNet50 training/validation loss curve.

Validation loss spikes sharply to above 1.0 at epoch 2, the same epoch where accuracy dropped in Figure 4, before early stopping halts training at epoch 6, well before the model reaches the stable, low-loss regime the other three baselines settle into. This early, forced stop is a direct consequence of the volatile validation behaviour and is a major reason ResNet50 finishes with the lowest Macro-F1.

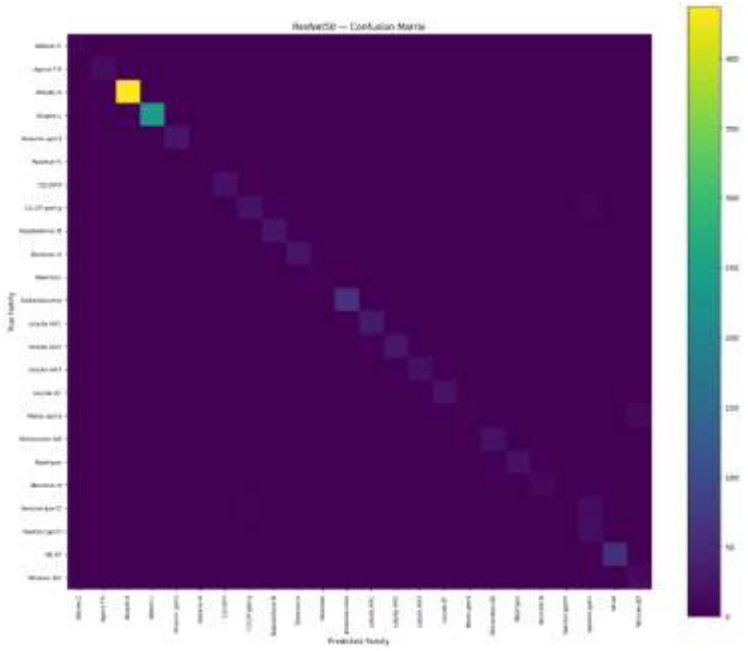

Figure 6: ResNet50 confusion matrix.

This matrix has visibly more off-diagonal spread than the Simple CNN's (Figure 3), particularly around Adialer.C, Malex.gen!J, and the Swizzor family pair, confirming numerically what the accuracy and loss curves already suggested: ResNet50's ImageNet features are tuned for natural-image textures (edges, colours, object parts), and malware byte-images are synthetic grayscale texture maps with a very different statistical structure, so a frozen natural-image backbone transfers less effectively here than a backbone trained from scratch on the target domain.

4.3 Baseline 3 - DenseNet121

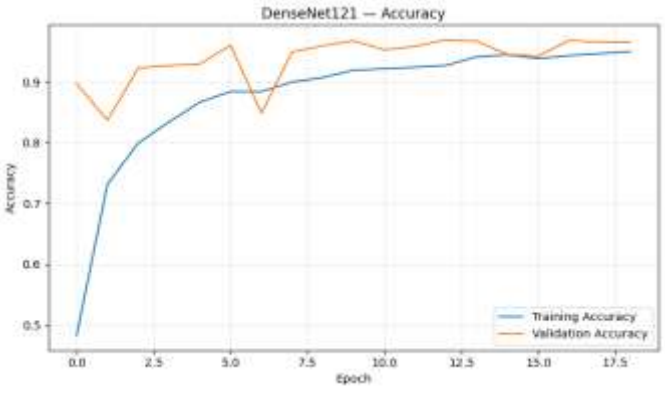

Figure 7: DenseNet121 training/validation accuracy curve.

DenseNet121 rises smoothly and reaches roughly 95% validation accuracy with far less epoch-to-epoch volatility than ResNet50, reflecting DenseNet's densely connected feature reuse producing more stable gradients even with a frozen backbone.

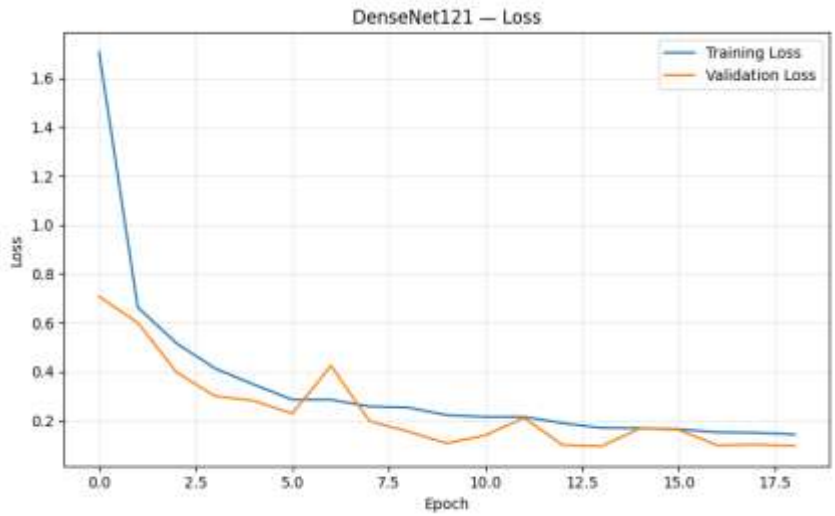

Figure 8: DenseNet121 training/validation loss curve.

Both losses fall steadily from a high starting point (training loss above 1.6) to below 0.2, with only a mild bump in validation loss around epoch 6. This smooth, well-behaved descent is consistent with DenseNet121 finishing as the second-strongest baseline (91.88% Macro-F1).

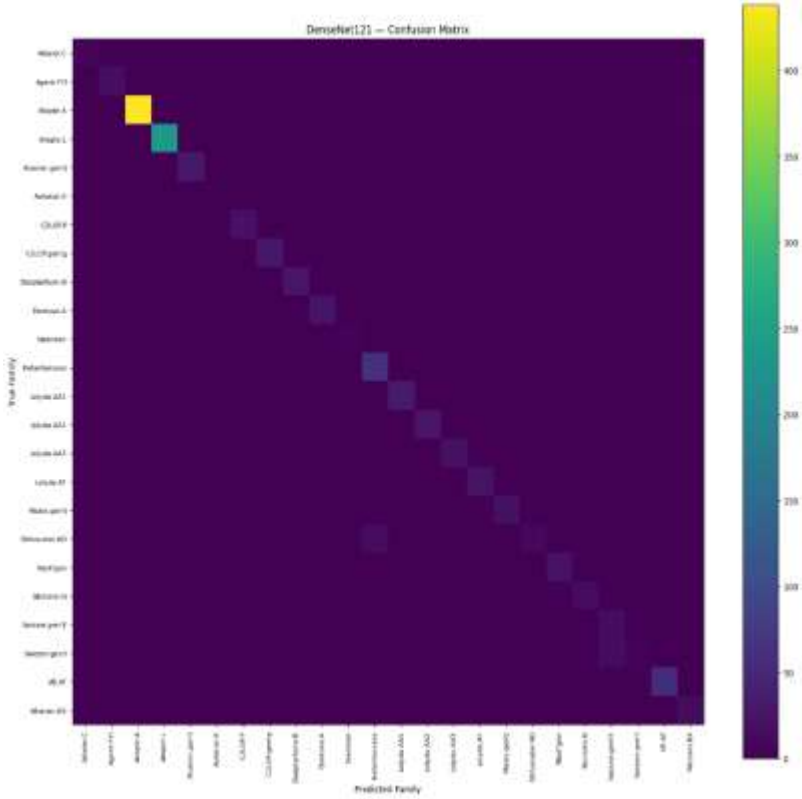

Figure 9: DenseNet121 confusion matrix.

The diagonal is cleaner than ResNet50's and close to the Simple CNN's, with only light confusion around the Swizzor.gen!E / Swizzor.gen!I pair and a few Allaple misclassifications, consistent with DenseNet121 reaching 96.18% accuracy at a noticeably higher parameter count and model size than the Simple CNN.

4.4 Baseline 4 - EfficientNetB0

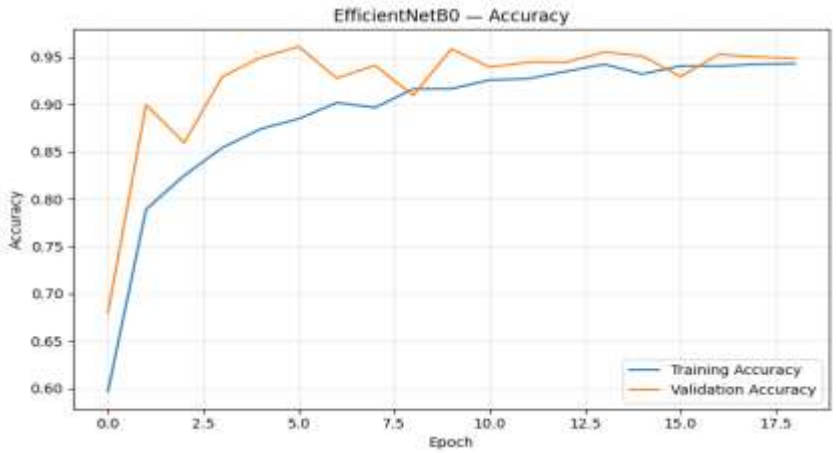

Figure 10: EfficientNetB0 training/validation accuracy curve.

EfficientNetB0 shows the best-behaved curve of the three pretrained backbones: validation accuracy leads training accuracy from the very first epoch and both climb steadily past 94%, with only minor fluctuation, before early stopping at epoch 19.

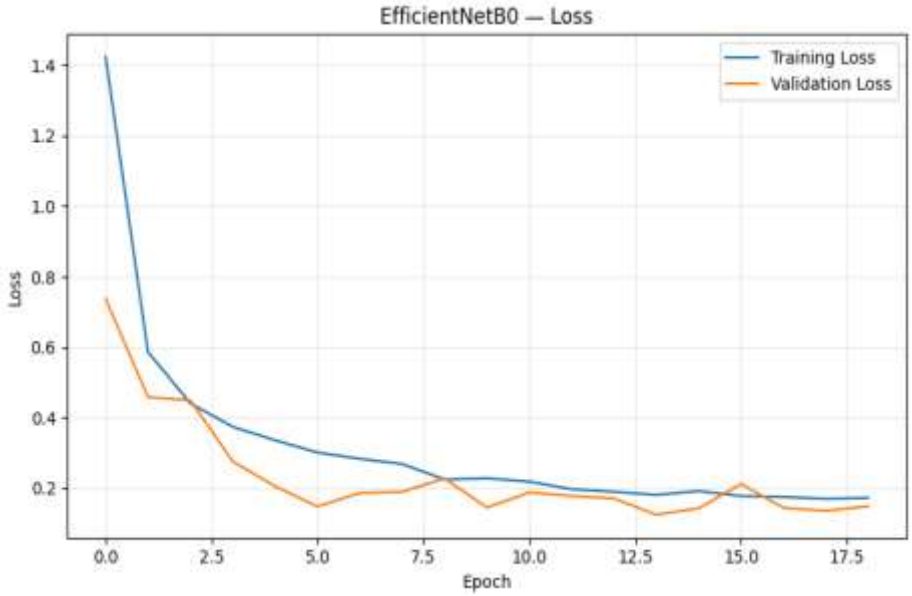

Figure 11: EfficientNetB0 training/validation loss curve.

Loss falls quickly and smoothly from an initial spike near 1.4 to under 0.2 within the first ten epochs and stays flat afterward, the most stable loss trajectory among all four baselines, matching EfficientNetB0's compound-scaled architecture [3] and explaining why it finishes as the strongest baseline overall.

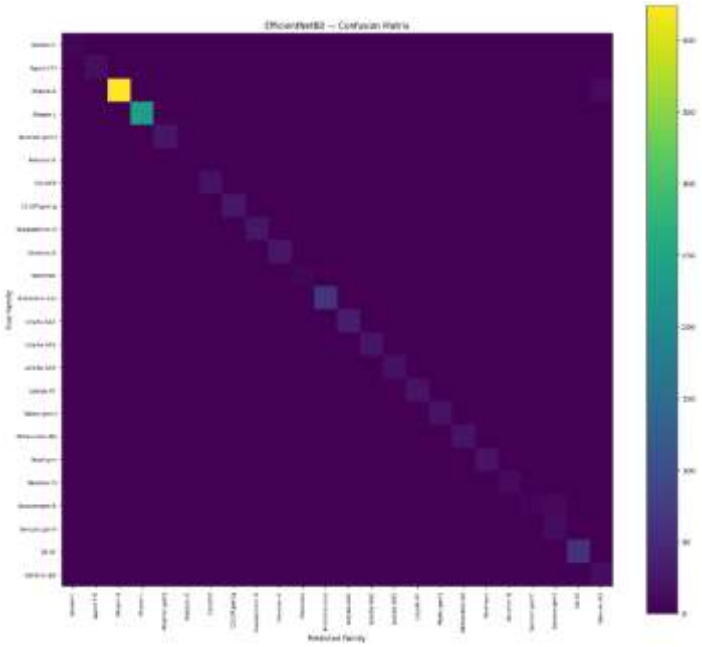

Figure 12: EfficientNetB0 confusion matrix the strongest baseline overall.

This is the cleanest confusion matrix of the four baselines: the diagonal is sharp across nearly every one of the 24 families, including small classes, with only a faint trace of confusion between Allaple.A and Wintrim.BX. This directly reflects EfficientNetB0's best-in-class Macro-F1 (92.96%) among the baselines.

4.5 Baseline Comparison

Table 3: Test-set performance and efficiency of the four CNN baselines, ranked as discussed below.

Model	Accuracy	Macro-F1	Weighted F1	Params	Size(MB)	Train time(min)	Inference(ma /img)
Simple CNN	96.26%	89.08%	95.81%	307,960	3.64	11.45	1.98
ResNet50 (frozen)	94.18%	80.79%	93.64%	23,853,080	93.67	2.95	7.59
DenseNet121 (frozen)	96.18%	91.88%	95.90%	7,171,800	29.85	10.13	11.18
EfficientNetB0 (frozen)	96.26%	92.96%	96.39%	4,216,635	18.18	5.43	6.02

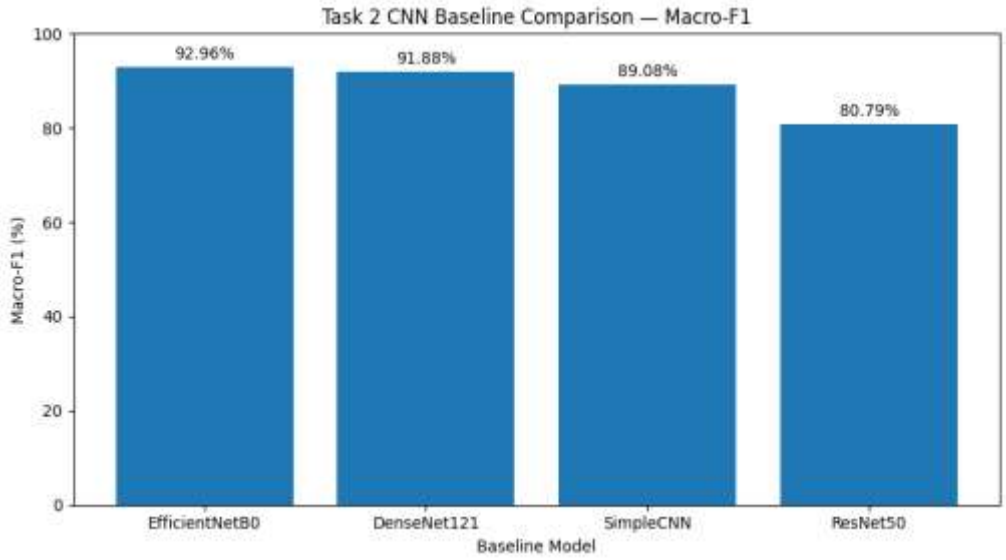

Figure 13: Macro-F1 across the four CNN baselines.

EfficientNetB0 is the strongest baseline by Macro-F1 (92.96%), followed by DenseNet121 (91.88%) and the lightweight Simple CNN (89.08%), with frozen ResNet50 last (80.79%). Notably, the Simple CNN with roughly 13× fewer parameters than EfficientNetB0 and 77× fewer than ResNet50 is already competitive with the pretrained backbones, which motivates building the attention mechanism on top of this compact backbone rather than on a much larger pretrained network.

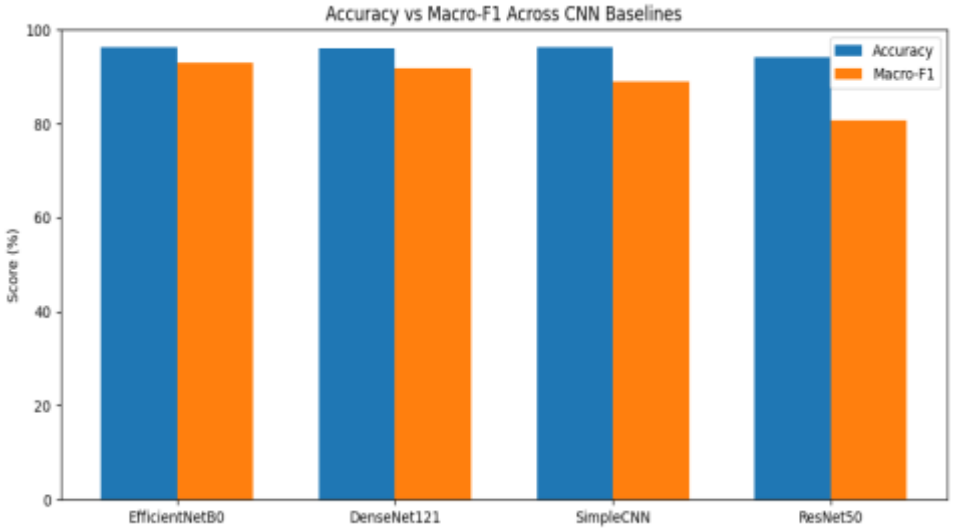

Figure 14: Accuracy vs. Macro-F1 across the four CNN baselines.

Placing accuracy next to Macro-F1 for each model makes the imbalance effect from Task 1 visible directly: every model's accuracy bar sits close to 94-96%, a narrow band that on its own would suggest the four baselines perform almost identically. The Macro-F1 bars tell a different story, spreading from 80.79% to 92.96%, because Macro-F1 weighs every one of the 24 families equally instead of being dominated by large classes like Allaple.A. The gap between the two bars is largest for ResNet50 (13.4 points) and smallest for EfficientNetB0 (3.3 points), meaning ResNet50's high accuracy is propped up disproportionately by the majority classes while EfficientNetB0's performance is more evenly spread across rare families as well. This is exactly why Macro-F1, not accuracy, is used as the ranking metric throughout this report.

5. Proposed Model: Custom CNN + CBAM

The research gap identified in Task 1 was that prior Malimg studies confound attention with other simultaneous changes (an added autoencoder, a second modality, a large pretrained backbone, GAN oversampling). The proposed model addresses this directly: it reuses the exact Simple CNN backbone and classifier head from Section 4.1, and inserts a single, well-defined attention module after the last convolutional block. Any performance difference between the two models can therefore be attributed to attention rather than to a confounded architectural change.

5.1 Proposed Architecture

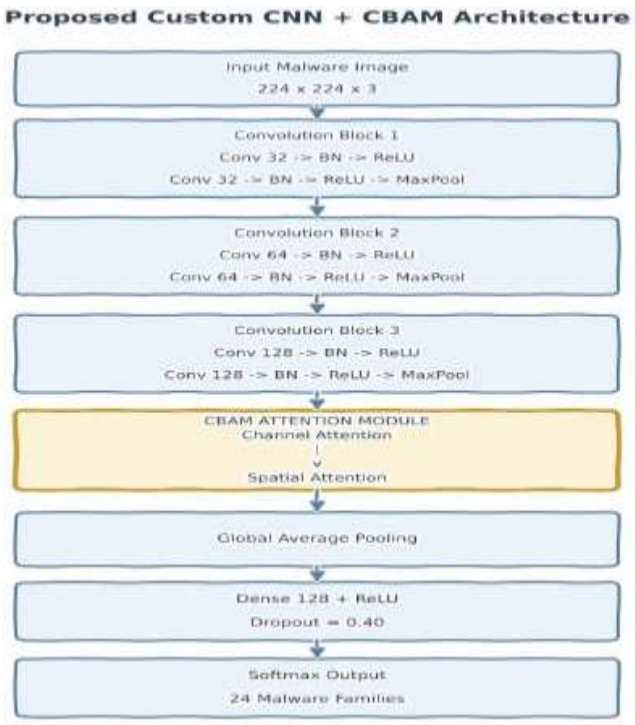

Figure 15: Proposed Custom CNN + CBAM architecture. CBAM is inserted after Convolution Block 3, before global average pooling, so it refines the deepest feature maps before classification.

The proposed network keeps the same three-block convolutional backbone as the Simple CNN baseline (32 → 64 → 128 filters), then inserts the CBAM attention module right after the last block, before the features are pooled and passed to the classifier. Placing CBAM here, rather than earlier in the network, lets it refine the richest, most semantically meaningful feature maps the backbone produces, immediately before the classification head decides on a malware family.

5.2 Attention Type and Placement

The chosen mechanism is CBAM (Convolutional Block Attention Module), a hybrid attention type combining channel attention and spatial attention applied sequentially. CBAM was selected because it lets channel and spatial attention be evaluated both jointly and (in the Task 3 ablation) independently, it adds a small number of parameters, and it is the mechanism used by two of the six related-work studies (Van Dao et al., 2022; Ayturan, 2026), which allows a direct, literature-grounded.

5.3 How CBAM Works

Channel attention determines which feature channels matter most. The 128 feature maps coming out of Block 3 are each summarized to a single number twice, once by global average pooling and once by global max pooling giving two 128-length vectors. Both vectors pass through the same small two-layer bottleneck (128 -> 16 -> 128, reduction ratio 8), and the two results are added and squashed with a sigmoid. This produces one weight per channel between 0 and 1, which is multiplied back onto the original feature maps, letting the network amplify the channels that carry discriminative malware texture and suppress the channels that mostly carry noise.

Spatial attention determines which regions of the image matter most. It compresses the channel-attended feature maps into two 2-D maps (an average and a max across all channels at every spatial location), stacks them, and passes them through a single 7×7 convolution with a sigmoid activation. The result is one weight per spatial location, which is multiplied back onto the feature maps so the network can focus more on the horizontal bands and dense texture blocks that Task 1's labelled-image analysis (Figures 2-3 of the Task 1 report) showed to be family-distinguishing regions, rather than treating every pixel location equally.

CBAM applies these two steps in sequence channel attention first, then spatial attention on the channel-refined output which is why the architecture diagram in Figure 8 shows a single combined block: channel refinement decides what to look at, and spatial refinement decides where to look, before the result is pooled and classified.

5.4 Proposed-Model Hyperparameters

Table 4: Architecture and training hyperparameters of the proposed Custom CNN + CBAM model.

Setting	Value
Backbone	3 conv blocks (32 → 64 → 128 filters), identical to the Simple CNN baseline
Attention module	CBAM — channel then spatial, inserted after Block 3
Channel reduction ratio	8
Spatial attention kernel	7 × 7
Classifier head	GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.40) → Softmax(24)
Optimizer / initial LR	Adam / 1e-3, with ReduceLROnPlateau
Class weights	Balanced (identical scheme to baselines)
Augmentation	Same conservative training-only policy as baselines
Total parameters	312,298 (vs. 307,960 for the no-attention Simple CNN — a +1.4% increase)

5.5 Training and Test Results

This section presents the proposed model's results in three parts, in the order they were produced: the learning curve from training, the confusion matrix from the held-out test set, and a summary table of all test-set metrics.

5.5.1 Learning Curve

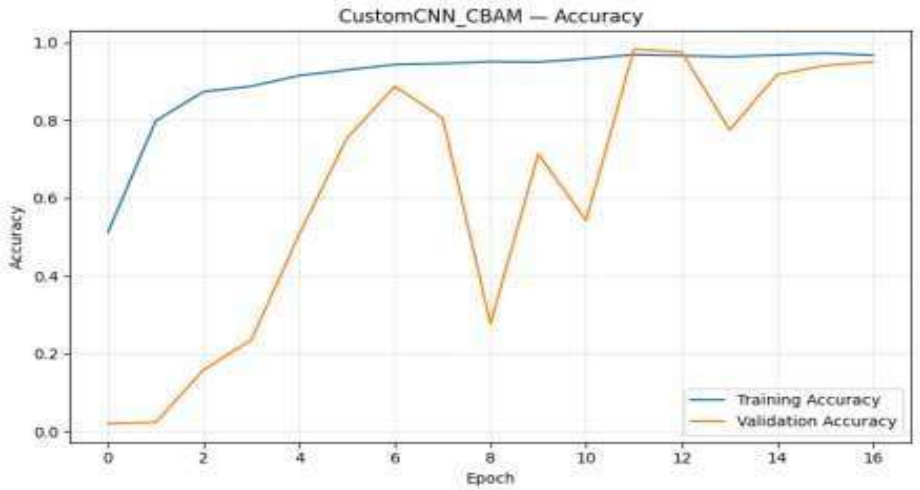

Figure 16: Custom CNN + CBAM training/validation accuracy curve.

Training accuracy rises smoothly and reaches above 95% within the first 10 epochs, which is expected since the training curve is not affected by class weighting the same way validation is. Validation accuracy is noisier in the early epochs (it drops as low as ~28% around epoch 8) because class weights push the model to correct rare-class errors aggressively before it has seen enough examples to do so reliably; this is the same instability visible in the Simple CNN's own curve (Figure 1) and is not specific to CBAM. From epoch 11 onward the validation curve stabilizes and tracks the training curve closely, which is why early stopping (patience 5, monitoring validation loss) selected a checkpoint from this later, stable region rather than from the noisy early epochs. Training finished after 17 epochs, 3 epochs earlier than the Simple CNN baseline, suggesting CBAM helped the model reach a good optimum slightly faster despite the extra attention computation per step.

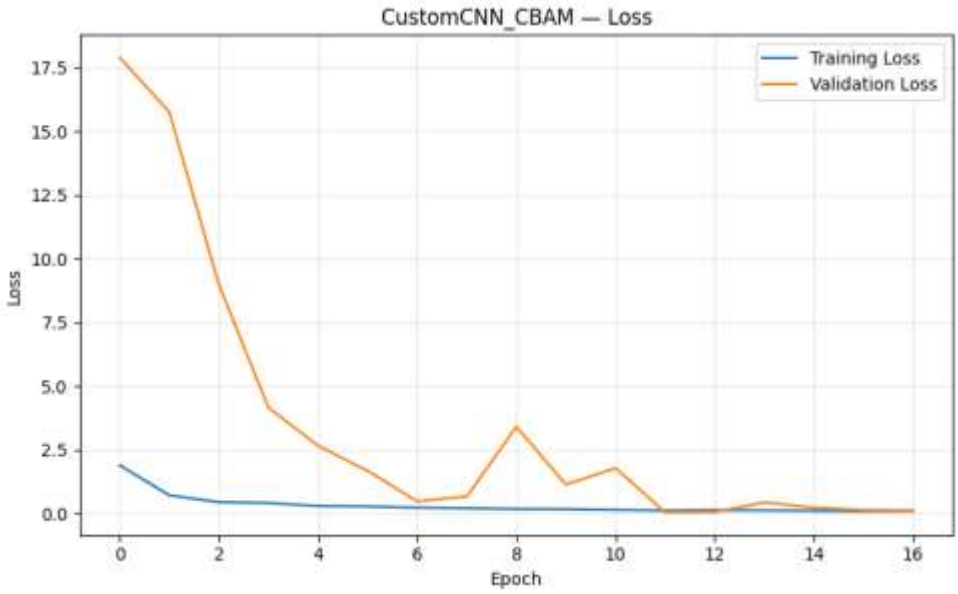

Figure 17. Custom CNN + CBAM training/validation loss curve.

Validation loss starts extremely high, close to 18 versus roughly 2 for training, and falls sharply within the first three epochs, a steeper initial drop than any baseline, then shows a smaller secondary bump around epoch 8, matching the same rare-class adjustment period visible in Figure 16. From epoch 11 onward, training and validation loss converge tightly near zero, mirroring the accuracy plateau and confirming the model reached a genuinely stable optimum rather than stopping early by chance.

5.5.2 Confusion Matrix

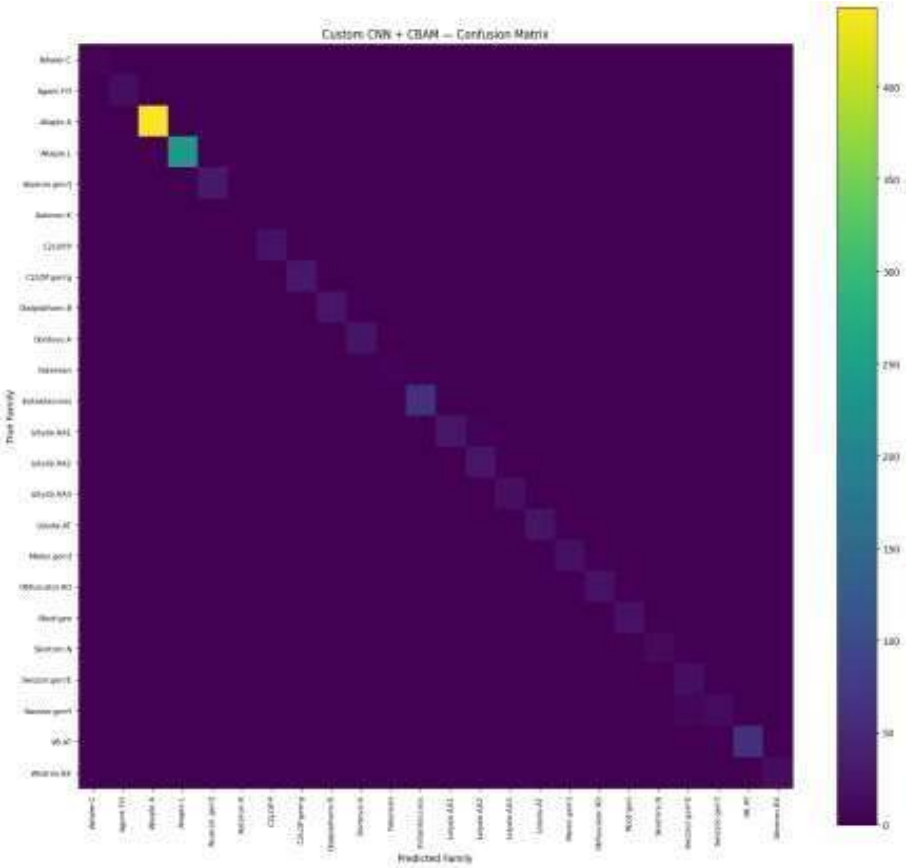

Figure 18: Custom CNN + CBAM confusion matrix the strongest model in Task 2.

The confusion matrix is almost entirely diagonal across all 24 malware families, including the smallest classes such as Skintrim.N and Autorun.K, which is the practical meaning behind the high Macro-F1 score: the model is not just accurate on the large classes (Allaple.A, Allaple.L) but is correctly separating the rare families as well. The few visible off-diagonal cells cluster around families that Task 1's labelled-image analysis already flagged as visually similar in raw texture (Section 5, Figures 2–3 of the Task 1 report) most notably the Swizzor.gen!E / Swizzor.gen!I pair, which remains the model's main source of confusion even with attention. This residual confusion is a direct target for the Grad-CAM and LIME explainability analysis in Task 3.

5.5.3 Performance Summary

Table 5: Custom CNN + CBAM test-set performance and efficiency summary.

Matric	Value
Accuracy	98.50%
Macro Precision	95.24%
Macro Recall	96.90%
Macro F1	95.72%
Weighted F1	98.50%
Training time	13.77 min (17 epochs)
Parameters	312,298
Model size	3.70 MB
Inference time	2.91 ms/image

Two gaps in this table are worth reading together. First, Macro Precision (95.24%) is noticeably lower than Macro Recall (96.90%), which means the model's remaining errors lean slightly toward false positives on a few families rather than missed detections consistent with the confusion pattern in Figure 10, where a handful of families draw predictions away from their visually similar neighbour rather than being missed altogether. Second, Weighted F1 (98.50%) is higher than Macro F1 (95.72%); since weighted F1 is dominated by the large classes (Allaple.A, Allaple.L) while Macro F1 weighs every family equally, this 2.78-point gap is the imbalance from Task 1 still showing through, and is exactly why Macro-F1 not accuracy or weighted F1 is used as the primary metric throughout this report. Training took 13.77 minutes for 17 epochs (48.6 s/epoch), close to the Simple CNN's 49 s/epoch, confirming that CBAM's computational overhead is negligible relative to the backbone.

6. Attention Effect: Direct Comparison

Because the proposed model reuses the Simple CNN backbone unchanged and adds only CBAM, the difference between the two results isolates the effect of attention on this dataset.

Table 6: Direct attention effect Simple CNN (no attention) vs. Custom CNN + CBAM, identical backbone.

Metric	Simple CNN(no attention)	Custom CNN + CBAM	Change
Accuracy	96.26%	98.50%	+2.24 pp
Macro Recall	92.29%	96.90%	+4.61 pp
Macro F1	89.08%	95.72%	+6.64 pp

Adding CBAM increased total parameters by only 1.4% (307,960 -> 312,298) yet produced the largest single improvement of any model change in Task 2: +6.64 percentage points of Macro-F1. This is the strongest direct evidence in the project so far that attention not extra capacity is responsible for the gain, since capacity was held almost constant. Macro recall also improved by 4.61 points, meaning the gain is not confined to the largest classes; the model is now correctly recognizing more of the rare families as well.

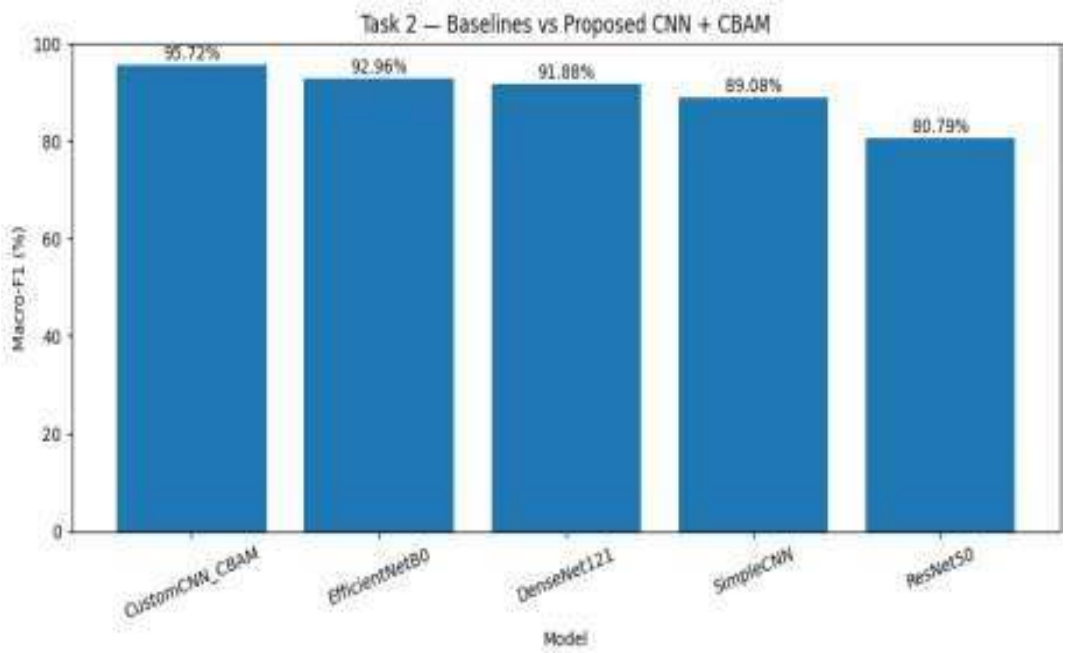

Figure 19: Final Task 2 comparison all four baselines and the proposed Custom CNN + CBAM model, ranked by Macro-F1.

Across all five Task 2 models, Custom CNN + CBAM is the best model by Macro-F1 (95.72%), ahead of EfficientNetB0 (92.96%, the best baseline) by 2.76 percentage points, while using roughly 13× fewer parameters and a model size of 3.70 MB versus 18.18 MB. In other words, the attention-augmented compact CNN outperforms every pretrained backbone tested while remaining the smallest and among the fastest models in the comparison.

7. Discussion

Taken together, the four baselines and the proposed model tell a consistent story about this dataset. The compact Simple CNN, with roughly 300 thousand parameters, already matched or outperformed two of the three much larger pretrained backbones, which suggests that Malimg's malware byte-images are better served by texture features learned directly from the domain than by features transferred from natural images. The frozen ResNet50 baseline made this point most clearly: despite its size, it finished last on Macro-F1, most likely because ImageNet's edge- and object-oriented filters do not transfer cleanly onto synthetic grayscale byte textures with no natural-image structure.

Adding CBAM to this same compact backbone produced the single largest improvement seen anywhere in Task 2. With only a 1.4% increase in parameters, Macro-F1 rose by 6.64 points and macro recall by 4.61 points over the no-attention Simple CNN, and the resulting model also surpassed every pretrained baseline, including EfficientNetB0, while staying the smallest model evaluated. Because the backbone and classifier head were held identical between the two runs, this gain can be attributed specifically to the channel-then-spatial refinement CBAM performs, rather than to any confound in capacity or training setup directly addressing the research gap identified in Task 1, where prior Malimg studies rarely isolated attention this cleanly.

The results are not without limitations worth noting here. All Task 2 numbers come from a single train/validation/test split, so the reported gains have not yet been checked for statistical stability across resampled splits. The confusion matrix also shows that a small number of visually similar families, most notably Swizzor.gen!E and Swizzor.gen!I, remain a persistent source of error even with attention, meaning CBAM narrowed the overall gap without fully resolving the hardest cases flagged during Task 1's labelled-image analysis. These two observations set the agenda for the next stage of the project: Task 3 will run 5-fold cross-validation with a significance test against the best baseline to confirm the Macro-F1 gap is not an artifact of one particular split, ablate CBAM's channel and spatial components separately to see how much of the gain each contributes on its own, and apply Grad-CAM and LIME to the Swizzor family confusions specifically to see what the model is, and is not, attending to when it gets them wrong.

8. Conclusion

Task 2 finalized a leakage-free data pipeline (8,018 deduplicated images across 24 evaluable families, verified with zero filepath and zero pixel-content overlap across partitions), trained and fully evaluated four CNN baselines under a shared protocol, and built the proposed Custom CNN + CBAM model on an identical backbone to the strongest baseline (Simple CNN). The proposed model achieved the best Macro-F1 in the project so far (95.72%), a 6.64-point improvement over its no-attention counterpart and a 2.76-point improvement over the best pretrained baseline, while remaining the most parameter-efficient model evaluated. These results directly answer Task 2's central question attention helps and provide the matched baseline needed for the ablation, explainability, and significance testing required in Task 3.

9. References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.

[2] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2017, pp. 4700-4708.

[3] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. Int. Conf. Machine Learning (ICML), 2019, pp. 6105-6114.

[4] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," in Proc. European Conf. Computer Vision (ECCV), 2018, pp. 3-19.

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[6] T. Van Dao, H. Sato, and M. Kubo, "An Attention Mechanism for Combination of CNN and VAE for Image-Based Malware Classification," IEEE Access, vol. 10, pp. 85127-85136, 2022.

[7] K. Ayturan, "A Hybrid Deep Learning Approach with Spatial Attention Mechanism for Visual-Based Malware Detection in IoT Security," Black Sea Journal of Engineering and Science, vol. 9, no. 3, pp. 1256-1268, 2026.

[8] Malimg Original, Kaggle dataset. Available: https://www.kaggle.com/datasets/ikrambenabd/malimg-original.